

Lab 02

OpenGL Basic Building Blocks: Triangles & Polygons

Objectives

The most basic building blocks of computer graphics and also OpenGL are **triangles** and **polygons**; this lab will explore how these can be used to build complex objects.

Special Instructions

READ these lab instructions BEFORE starting!

1. In Lab01, you were given 4 files – CGLabmain.cpp, CGLabmain.hpp, CGLab01.cpp and CGLab01.hpp files. We still need the first two files in this lab, but we will create the other NEW files on top of that later.
2. DO NOT copy-and-paste blindly! Some portions of code provided are just modifications, or additions of some lines of code. Always check carefully WHAT and WHERE to copy and paste. Most modifications/additions are in **BOLD**.

Programming Exercises

(100 minutes)

Part 1 – Back-Face Culling & Vertex Ordering

Task 1A : Understanding “front face” and “back face” (10 minutes)

Run the CGLabs .cbp project again; remember in Lab01, you were asked to look under the pyramid? Ok, let's do that again, what do you see?

There appear to be a problem right? No, actually it was intentionally programmed to be that way in order to demonstrate the idea of **front face** and **back face**.

In our pyramid (formed by triangles) example, let's suppose we add a base to it, making it a solid-looking pyramid, we will never look from the inside of the pyramid (unless there is a door or a hole or without base). Then we will always see only one side of the triangle, this side is called the **front face**, the part that we will never see is called the **back face**.

Normally, triangles are used to build solid objects, thus OpenGL by default only renders (draws) the front face and not the back face. If the front face is not facing us (the viewers), it means the back face is facing us. However, since it is a solid object, we will never get to see the back face. In actual fact, we will never get to see that triangle, thus we should just ignore that triangle. The process of ignoring (not process for rendering) a triangle (or a face) when the back face is facing our direction (but blocked from our views by other part of the object) is called **Back-Face Culling**. This process will make the rendering more efficient (i.e. faster).

However, there are exceptions, in cases such as our example where we are not having a solid object, we would like to be able to see both the front face and back face. In this case, we would like to disable the back-face culling, to do that modify the `void SimpleTriangles::draw()` function as below:

```
void SimpleTriangles::draw()
```

```

{
    glDisable(GL_CULL_FACE); //disable face culling
    .....
    .....
    .....
    glEnable(GL_CULL_FACE); //restore back the default setting
}

```

After adding these lines, try running it again and look from below the pyramid, what do you see?

Task 1B : Understanding “clock-wise” and “counter-clock-wise” (15 minutes)

Wait a minute, HOW does the computer know where is front-face and where is back-face?

Now, look carefully again at the `void SimpleTriangles::draw()` function, as shown below:

```

void SimpleTriangles::draw()
{
    glDisable(GL_CULL_FACE); //disable face culling

    glBegin(GL_TRIANGLES);
        glColor3f(0.0f, 0.0f, 1.0f ); glVertex3f( 0.0f, 5.0f, 0.0f );
        glColor3f(1.0f, 0.0f, 0.0f ); glVertex3f(-5.0f, -5.0f, 5.0f );
        glColor3f(0.0f, 1.0f, 0.0f ); glVertex3f( 5.0f, -5.0f, 5.0f );

        glColor3f(0.0f, 0.0f, 1.0f ); glVertex3f( 0.0f, 5.0f, 0.0f );
        glColor3f(0.0f, 1.0f, 0.0f ); glVertex3f( 5.0f, -5.0f, 5.0f );
        glColor3f(0.0f, 1.0f, 1.0f ); glVertex3f( 5.0f, -5.0f, -5.0f );

        glColor3f(0.0f, 0.0f, 1.0f ); glVertex3f( 0.0f, 5.0f, 0.0f );
        glColor3f(0.0f, 1.0f, 1.0f ); glVertex3f( 5.0f, -5.0f, -5.0f );
        glColor3f(1.0f, 1.0f, 0.0f ); glVertex3f(-5.0f, -5.0f, -5.0f );

        glColor3f(0.0f, 0.0f, 1.0f ); glVertex3f( 0.0f, 5.0f, 0.0f );
        glColor3f(1.0f, 1.0f, 0.0f ); glVertex3f(-5.0f, -5.0f, -5.0f );
        glColor3f(1.0f, 0.0f, 0.0f ); glVertex3f(-5.0f, -5.0f, 5.0f );
    glEnd();

    glEnable(GL_CULL_FACE); //restore back the default setting
}

```

The coordinate of the triangles are specified in between these two lines:

```

    glBegin(GL_TRIANGLES);
    ...
    glEnd();

```

The `glColor3f(..., ..., ...)`; are used to specify the red, green, blue (also called RGB) color components of all the points created after this statement. A point is also called a vertex; many points are called vertices (sometime “vertexes”).

The `glVertex3f(..., ..., ...)`; are the actual statements used to specify the (x,y,z) coordinate of a vertex. In this case, the first 3 vertices will form the first triangle; the second set of three vertices will form the second triangle and so on.

What do you notice about the way the 3 vertices of a triangle are specified?



Question: Are the sequence of the 3 vertices of each triangles specified in a clock-wise (CW) manner, or are they specified in a counter-clock-wise (CCW) manner?

Let's enable back-face culling again by simply commenting out the 2 lines related to GL_CULL_FACE, and let's change the sequence of the three vertices of the second triangle and also the last triangle as highlighted below:

```
void SimpleTriangles::draw()
{
    // glDisable(GL_CULL_FACE); //disable face culling
    glBegin(GL_TRIANGLES);
        glColor3f(0.0f, 0.0f, 1.0f ); glVertex3f( 0.0f, 5.0f, 0.0f );
        glColor3f(1.0f, 0.0f, 0.0f ); glVertex3f(-5.0f, -5.0f, 5.0f );
        glColor3f(0.0f, 1.0f, 0.0f ); glVertex3f( 5.0f, -5.0f, 5.0f );

        glColor3f(0.0f, 0.0f, 1.0f ); glVertex3f( 0.0f, 5.0f, 0.0f );
        glColor3f(0.0f, 1.0f, 1.0f ); glVertex3f( 5.0f, -5.0f, -5.0f );
        glColor3f(0.0f, 1.0f, 0.0f ); glVertex3f( 5.0f, -5.0f, 5.0f );

        glColor3f(0.0f, 0.0f, 1.0f ); glVertex3f( 0.0f, 5.0f, 0.0f );
        glColor3f(0.0f, 1.0f, 1.0f ); glVertex3f( 5.0f, -5.0f, -5.0f );
        glColor3f(1.0f, 1.0f, 0.0f ); glVertex3f(-5.0f, -5.0f, -5.0f );

        glColor3f(0.0f, 0.0f, 1.0f ); glVertex3f( 0.0f, 5.0f, 0.0f );
        glColor3f(1.0f, 0.0f, 0.0f ); glVertex3f(-5.0f, -5.0f, 5.0f );
        glColor3f(1.0f, 1.0f, 0.0f ); glVertex3f(-5.0f, -5.0f, -5.0f );
    glEnd();

    // glEnable(GL_CULL_FACE); //restore back the default setting
}
```

Now rotate the pyramid, try looking from the top as well as looking from the bottom, what is the effect? Why?



Question: The front face is defined as the side of the triangle when viewed by a viewer from that side. The vertices are specified in the: (choose the answer)

- A. Clock-Wise (CW) manner
- B. Counter- Clock-Wise (CCW) manner

You can also specify the front face to use clock-wise (CW) vertex ordering or counter-clock-wise (CCW) vertex ordering by setting `glFrontFace(GL_CW)` or `glFrontFace(GL_CCW)` respectively.

You may refer to the OpenGL Reference Manual (<https://www.opengl.org/sdk/docs/man/>) to find out more on how to use these functions.

Part 2 – Basic Building Blocks: Creating Triangles (30 minutes)

Now, we are ready to explore how to create our own models based on triangles.

For this lab, we are about to create and add two new files into our project. We no longer need to use the CGLab01.hpp and CGLab01.cpp anymore and you could choose to remove these two files from your project (right-click at the file name in the project panel and select 'Remove file from project', this will not delete the actual files). You could also choose to keep these files in your project as it will not conflict with the new files to be added. We are using the namespace to prevent the clashing of variable and class names.

Now, we are ready to create two new files for this lab, the new files to be created are CGLab02.hpp and CGLab02.cpp. To create a new file and add it into the CodeBlocks project, select File->New->Empty file and type in the filename for the new file, then answer 'Yes' when you are being asked "Do you want to add the new file in the active project?" (Alternatively, you can also select File->New->File... and follow the steps using the wizard, but this way is a bit more tedious.)

The content of the CGLab02.hpp and CGLab02.cpp are shown below, cut-and-paste them into the new files that you have just created.

Below is the CGLab02.hpp:

```
/*
TCS2221 Computer Graphics
CGLab02.hpp
Objective: Header File for Lab02 on triangles, quads and polygons
*/

#ifndef YP_CGLAB02_HPP
#define YP_CGLAB02_HPP

#include "CGLabmain.hpp"

namespace CGLab02 {

class MyTriangles
{
public:
    void draw();
};

//-----
//the main program will call methods from this class
class MyVirtualWorld
{
public:
    MyTriangles mytriangles;
    void draw()
    {
        mytriangles.draw();
    }
    void tickTime()
    {
        //do nothing, reserved for doing animation
    }

    //for any one-time only initialization of the
```

```

    //    virtual world before any rendering takes place
    //    BUT after OpenGL has been initialized
    void init()
    {
    }

};

}; //end of namespace CGLab02

#endif //YP_CGLAB02_HPP

```

Below is the CGLab02.cpp:

```

/*
TCS2221 Computer Graphics
CGLab02.cpp
Objective: Lab02 on Triangles, Quads and Polygons
*/

#include <GL/glut.h>
#include "CGLab02.hpp"

using namespace CGLab02;

//define the draw() function of MyTriangles
void MyTriangles::draw()
{
    glDisable(GL_CULL_FACE);

    glBegin(GL_TRIANGLES);
        glColor3f(0.0f, 0.0f, 1.0f );   glVertex3f( 0.0f, 5.0f, 0.0f );
        glColor3f(1.0f, 0.0f, 0.0f );   glVertex3f(-5.0f, -5.0f, 5.0f );
        glColor3f(0.0f, 1.0f, 0.0f );   glVertex3f( 5.0f, -5.0f, 5.0f );

        glColor3f(0.0f, 0.0f, 1.0f );   glVertex3f( 0.0f, 5.0f, 0.0f );
        glColor3f(0.0f, 1.0f, 0.0f );   glVertex3f( 5.0f, -5.0f, 5.0f );
        glColor3f(0.0f, 1.0f, 1.0f );   glVertex3f( 5.0f, -5.0f, -5.0f );

        glColor3f(0.0f, 0.0f, 1.0f );   glVertex3f( 0.0f, 5.0f, 0.0f );
        glColor3f(0.0f, 1.0f, 1.0f );   glVertex3f( 5.0f, -5.0f, -5.0f );
        glColor3f(1.0f, 1.0f, 0.0f );   glVertex3f(-5.0f, -5.0f, -5.0f );

        glColor3f(0.0f, 0.0f, 1.0f );   glVertex3f( 0.0f, 5.0f, 0.0f );
        glColor3f(1.0f, 1.0f, 0.0f );   glVertex3f(-5.0f, -5.0f, -5.0f );
        glColor3f(1.0f, 0.0f, 0.0f );   glVertex3f(-5.0f, -5.0f, 5.0f );
    glEnd();

    glEnable(GL_CULL_FACE);
}

```

After creating the two files above, modify two lines in CGLabmain.cpp so that these two lines,

```

...
#include "CGLab01.hpp"

```

```
using CGLab01::MyVirtualWorld;
...
```

will be changed to...

```
...
#include "CGLab02.hpp"

using CGLab02::MyVirtualWorld;
...
```

The purpose of this modification is to ensure that the `MyVirtualWorld` class that we wish to use is from the namespace called `CGLab02` which is being defined in file `CGLab02.cpp`. This is particular useful when we have a few version of the same class exist in a project with defined in separate namespaces. In our example, the `MyVirtualWorld` class in Lab01 is defined in the `CGLab01` namespace while the `MyVirtualWorld` class in Lab02 is defined in the `CGLab02` namespace.

Now run the project again and you will see the same pyramid as in Lab01 now sit at the center of the screen without the rest of the objects. The origin of the world coordinate ($x=0, y=0, z=0$) is actually at the center of the pyramid, you can verify that by tracing out the coordinates of the vertices. The x-axis goes from left (negative-x) to right (positive-x), the y-axis goes from bottom (negative-y) to top (positive-y) and the z-axis goes from the back of the screen (negative-z) to the front of the screen (positive-z), thus the viewer is looking at the virtual world from the positive-z position and looking towards the negative-z. This coordinate system is actually the world coordinate system as programmed in this program, it is not necessary the default setting.

Try to understand the overall flow of the program execution. Note that the `draw()` function of class `MyTriangles` is called by the `draw()` function of class `MyVirtualWorld` which in turn is being called from `myDisplayFunc()` function of the main program (in `CGLabmain.cpp`).

Try using only one color for each triangle by modifying the code in `CGLab02.cpp` as below:

```
....
glBegin(GL_TRIANGLES);
    glColor3f(1.0f, 0.0f, 0.0f ); glVertex3f( 0.0f, 5.0f, 0.0f );
    // glColor3f(1.0f, 0.0f, 0.0f );
    glVertex3f(-5.0f, -5.0f, 5.0f );
    // glColor3f(0.0f, 1.0f, 0.0f );
    glVertex3f( 5.0f, -5.0f, 5.0f );

    glColor3f(0.0f, 1.0f, 0.0f ); glVertex3f( 0.0f, 5.0f, 0.0f );
    // glColor3f(0.0f, 1.0f, 0.0f );
    glVertex3f( 5.0f, -5.0f, 5.0f );
    // glColor3f(0.0f, 1.0f, 1.0f );
    glVertex3f( 5.0f, -5.0f, -5.0f );

    glColor3f(0.0f, 0.0f, 1.0f ); glVertex3f( 0.0f, 5.0f, 0.0f );
    // glColor3f(0.0f, 1.0f, 1.0f );
    glVertex3f( 5.0f, -5.0f, -5.0f );
    // glColor3f(1.0f, 1.0f, 0.0f );
    glVertex3f(-5.0f, -5.0f, -5.0f );

    glColor3f(1.0f, 0.0f, 1.0f ); glVertex3f( 0.0f, 5.0f, 0.0f );
    // glColor3f(1.0f, 1.0f, 0.0f );
```

```

    glVertex3f(-5.0f, -5.0f, -5.0f );
    // glColor3f(1.0f, 0.0f, 0.0f );
    glVertex3f(-5.0f, -5.0f, 5.0f );
    glEnd();
    ....

```

What did you observe about the colors this time?

Part 3 – Basic Building Blocks: Quads & Polygons

Task 3A : Using Quads (30 minutes)

Earlier, we have learned how to use triangles to build objects; triangles are special type of polygons where three vertices form a triangle. Triangles are special as a triangle is guaranteed to be flat (coplanar).

Before we go into polygons in general, let's us explore another special type of polygon called **Quads** (i.e. quadrilaterals). Quads are simply a polygon with four vertices, most of the time it is used to build squares or rectangles or the faces of a 3D box. However, it is not limited to this only; it can be any objects with faces formed by four-sided polygons.

This time we want to build a simple world. How about a room with walls, with a table, a dustbin, and some shelves?

Just as mentioned above, create a new class called `MyRoom` in the same files and declare some functions that we need to draw the items we want to build. Also, create an instance of the `MyRoom` class object in the `MyVirtualWorld` class, and call its `draw()` function as well.

Additions for `CGLab02.hpp` are as shown below:

```

...
...
namespace CGLab02 {

class MyTriangles
{
public:
    void draw();
};

class MyRoom
{
public:
    void draw();
    void drawWalls();
    void drawTable();
    void drawDustbin();
    void drawShelves();
};

//-----
//the main program will call methods from this class
class MyVirtualWorld

```

```

{
public:
    MyTriangles mytriangles;
    MyRoom myroom;

    void draw()
    {
        mytriangles.draw();
        myroom.draw();
    }
    void tickTime()
    {
        //do nothing, reserved for doing animation
    }
    ...
}

```

IMPORTANT: Since we want to work on building the room objects now, let's “**turn off**” the drawing of the earlier triangles temporarily by commenting out the function call `mytriangles.draw()`. This is a good way to work on each individual object without being cluttered with other objects in the world that you have created earlier.

```

...

class MyVirtualWorld
{
public:
    MyTriangles mytriangles;
    MyRoom myroom;

    void draw()
    {
        // mytriangles.draw();
        myroom.draw();
    }
    ...
}

```

Then, function implementations of the `MyRoom` class are added to `CGLab02.cpp`, as shown below:

```

...
...
using namespace CGLab02;

void MyTriangles::draw()
{
    ...
    ...
    ...
}

```



```
void MyRoom::draw()
{
    drawWalls();
    drawTable();
    drawDustbin();
    drawShelves();
}

void MyRoom::drawWalls()
{
    // fill this up later
}

void MyRoom::drawTable()
{
    // fill this up later
}

void MyRoom::drawDustbin()
{
    // fill this up later
}

void MyRoom::drawShelves()
{
    // fill this up later
}
```

NOTE: There's no need to comment out the function implementations of the `MyTriangles` class. Now we shall try to create our virtual room. Let's fill up the `void MyRoom::drawWalls()` function as follows:

```
void MyRoom::drawWalls()
{
    glDisable(GL_CULL_FACE); //this is needed

    glBegin(GL_QUADS);
        //back wall
        glColor3f(0.5f, 0.1f, 0.3f );
        glVertex3f(-25.1f, 15.1f, -5.1f );
        glVertex3f(-25.1f, -0.1f, -5.1f );
        glVertex3f( 25.1f, -0.1f, -5.1f );
        glVertex3f( 25.1f, 15.1f, -5.1f );
        //left wall
        glColor3f(0.5f, 0.1f, 0.8f );
        glVertex3f(-25.1f, 15.1f, 15.1f );
        glVertex3f(-25.1f, -0.1f, 15.1f );
        glColor3f(0.3f, 0.8f, 0.1f );
        glVertex3f(-25.1f, -0.1f, -5.1f );
        glVertex3f(-25.1f, 15.1f, -5.1f );
        //floor
        // fill this up to draw the floor
    glEnd();

    glEnable(GL_CULL_FACE); //restore to default
}
```

Now run the program and you will see two rectangles on the screen, we will assume this is two walls of the room for now. Take note, we must disable the back-face culling by calling `glDisable(GL_CULL_FACE)` if we wish to see properly the back of this wall. **It is a good practice to disable back-face culling whenever you are building the objects.**

Here you can see that to draw a four-sided polygon (quads), it is pretty similar to the way we draw triangles. All we need to do is to specify sets of four vertices (for each quads) in between the `glBegin(GL_QUADS);` and `glEnd();`

VERY IMPORTANT NOTE: `glBegin(...);` and `glEnd();` must always come together, if you miss out anyone of these, you will get very weird results on your screen!

Now let's build a table, we will fill up the `void MyRoom::drawTable()` function as below:

```
void MyRoom::drawTable()
{
    glDisable(GL_CULL_FACE); //disable this while creating the object

    //remember : specify in counter-clock-wise
    //do incrementally

    glBegin(GL_QUADS);
        //table top
        //top
        glColor3f(1.0f, 1.0f, 0.5f );
        glVertex3f( -10.0f, 7.0f, 5.0f );
        glVertex3f( 10.0f, 7.0f, 5.0f );
        glVertex3f( 10.0f, 7.0f, -5.0f );
        glVertex3f( -10.0f, 7.0f, -5.0f );
        //bottom
        glVertex3f( -10.0f, 6.0f, 5.0f );
        glVertex3f( -10.0f, 6.0f, -5.0f );
        glVertex3f( 10.0f, 6.0f, -5.0f );
        glVertex3f( 10.0f, 6.0f, 5.0f );
        //front
        glVertex3f( -10.0f, 7.0f, 5.0f );
        glVertex3f( -10.0f, 6.0f, 5.0f );
        glVertex3f( 10.0f, 6.0f, 5.0f );
        glVertex3f( 10.0f, 7.0f, 5.0f );
        //back
        glVertex3f( -10.0f, 7.0f, -5.0f );
        glVertex3f( 10.0f, 7.0f, -5.0f );
        glVertex3f( 10.0f, 6.0f, -5.0f );
        glVertex3f( -10.0f, 6.0f, -5.0f );
        //table left leg
        //left
        glColor3f(0.5f, 0.5f, 1.0f );
        glVertex3f( -11.0f, 7.0f, -5.0f );
        glVertex3f( -11.0f, 0.0f, -5.0f );
        glVertex3f( -11.0f, 0.0f, 5.0f );
        glVertex3f( -11.0f, 7.0f, 5.0f );
        //right
        glVertex3f( -10.0f, 7.0f, -5.0f );
        glVertex3f( -10.0f, 7.0f, 5.0f );
        glVertex3f( -10.0f, 0.0f, 5.0f );
        glVertex3f( -10.0f, 0.0f, -5.0f );
        //front
```

```

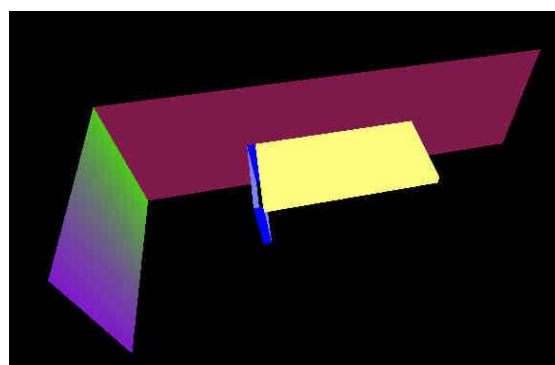
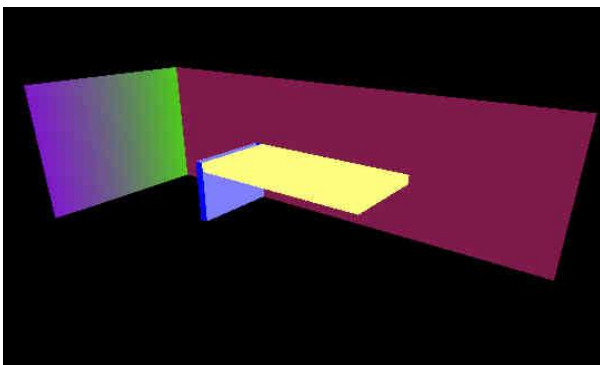
    glColor3f(0.0f, 0.0f, 1.0f );
    glVertex3f( -11.0f,  7.0f,  5.0f );
    glVertex3f( -11.0f,  0.0f,  5.0f );
    glVertex3f( -10.0f,  0.0f,  5.0f );
    glVertex3f( -10.0f,  7.0f,  5.0f );
    //back
    glVertex3f( -11.0f,  7.0f, -5.0f );
    glVertex3f( -10.0f,  7.0f, -5.0f );
    glVertex3f( -10.0f,  0.0f, -5.0f );
    glVertex3f( -11.0f,  0.0f, -5.0f );
    //top
    glVertex3f( -11.0f,  7.0f, -5.0f );
    glVertex3f( -11.0f,  7.0f,  5.0f );
    glVertex3f( -10.0f,  7.0f,  5.0f );
    glVertex3f( -10.0f,  7.0f, -5.0f );
    //bottom
    glVertex3f( -11.0f,  0.0f, -5.0f );
    glVertex3f( -10.0f,  0.0f, -5.0f );
    glVertex3f( -10.0f,  0.0f,  5.0f );
    glVertex3f( -11.0f,  0.0f,  5.0f );
    // fill this up to complete the table
    glEnd();

    glEnable(GL_CULL_FACE); //restore to default
}

```

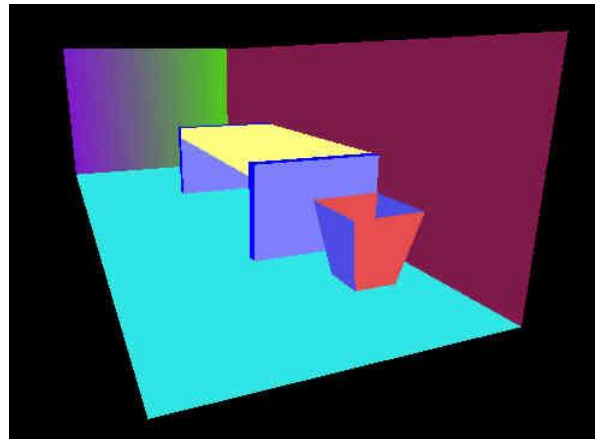
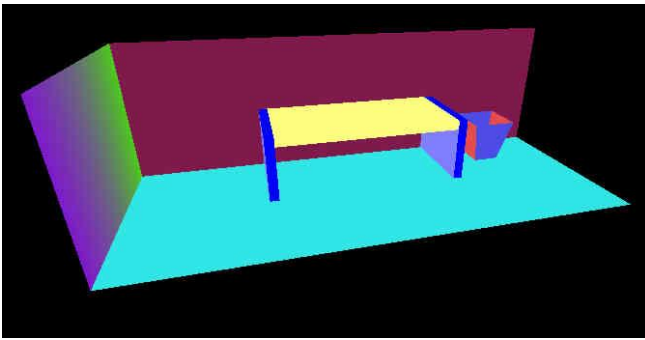
For solid (or complex) objects that are not thin or face-like, such as the table, we normally do NOT disable the back-face culling because it will be more efficient to allow the default back-face culling. However, while building the object, it would be useful to disable that so that you can have a full view of all the polygons being created. You don't want to miss out any sides!

Now if you run the program, you should see something like this from two different views:



Task 3B : Complete the furniture! (Homework)

Your task now is to complete the room by finishing up the table legs, add the floor and add a dustbin to the room (we will do the shelves a bit later). Your finished product should look something like this below (seen from two different views):



Important Notes:

- Create your objects incrementally, that is test run your program after you have created each object.
- REMEMBER! The sequence of the vertices for each polygon must be specified in the counter-clock-wise (CCW) manner, it is counter-clock-wise with regards to being viewed from the outside of the object (that is, the front, viewable face of the object).

Task 3C : Creating general polygons (15 minutes)

Now that you are able to create triangles and quads, it is time to learn to create general polygons with any number of sides. However, one VERY IMPORTANT restriction is that your polygons must be convex and flat (coplanar)! You may still be able to create non-convex or non-flat polygons, but the results would be unpredictable.

To create polygon, EACH polygon must be specified within the `glBegin(GL_POLYGON);` and `glEnd();` We will demonstrate this by filling up our `void MyRoom::drawShelves()` function as below:

```
void MyRoom::drawShelves()
{
    glDisable(GL_CULL_FACE); //this is needed

    glBegin(GL_POLYGON);
        glColor3f(1.0f, 1.0f, 0.2f );
        glVertex3f( -25.0f, 2.0f, -5.0f );
        glVertex3f( -25.0f, 2.0f, 3.0f );
        glVertex3f( -21.0f, 2.0f, 3.0f );
        glVertex3f( -17.0f, 2.0f, -1.0f );
        glVertex3f( -17.0f, 2.0f, -5.0f );
    glEnd();

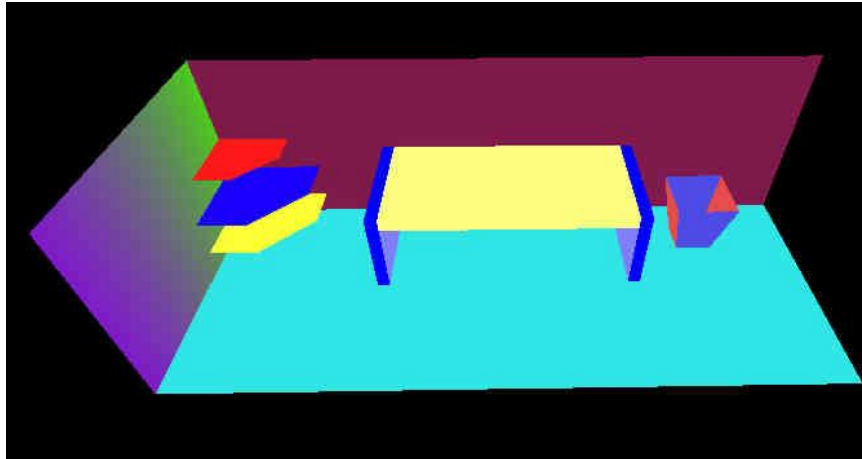
    glBegin(GL_POLYGON);
        glColor3f(0.1f, 0.0f, 1.0f );
        glVertex3f( -25.0f, 5.0f, -5.0f );
        glVertex3f( -25.0f, 5.0f, 3.0f );
        glVertex3f( -21.0f, 5.0f, 3.0f );
        glVertex3f( -17.0f, 5.0f, -1.0f );
        glVertex3f( -17.0f, 5.0f, -5.0f );
    glEnd();

    // fill this up to complete the shelves
}
```

```
glEnable(GL_CULL_FACE); //restore to default  
}
```

NOTE: One major difference with triangles or quads is that EACH polygon must be enclosed within one pair of `glBegin(GL_POLYGON);` and `glEnd();` This is because a polygon can consist of any number of vertices, and we need to indicate when it ends.

Your task now is to complete the shelves so that your room looks something like this:



Try and add more objects inside. Your objects can be anything built using triangles, quads, polygons, or a combination of those types.

How about putting something on the wall? ☺

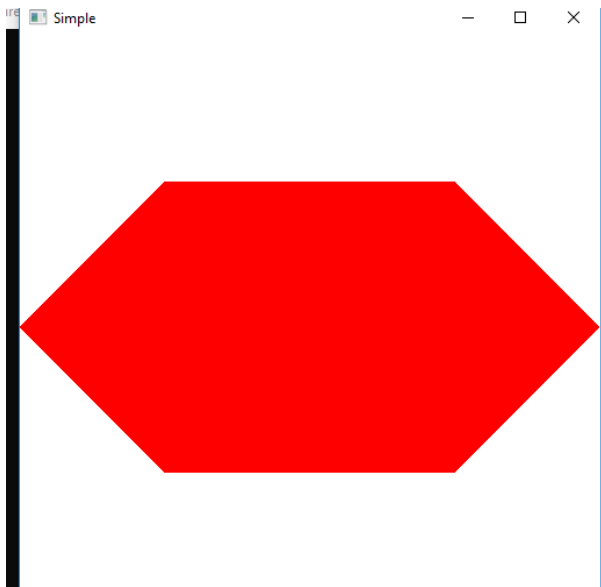
Extra Exercises

VERY IMPORTANT

BEFORE coming to the next lab, you **MUST**:

1. **COMPLETE** all the exercises in this lab!
2. **DO** the extra exercises below (if there are any)!
3. **READ** the lab instructions!
4. **PREPARE** files for next lab!

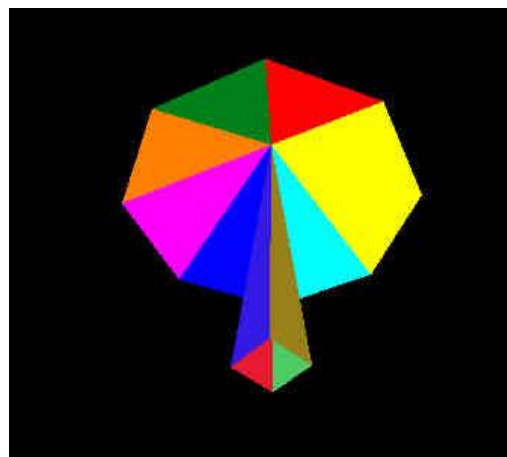
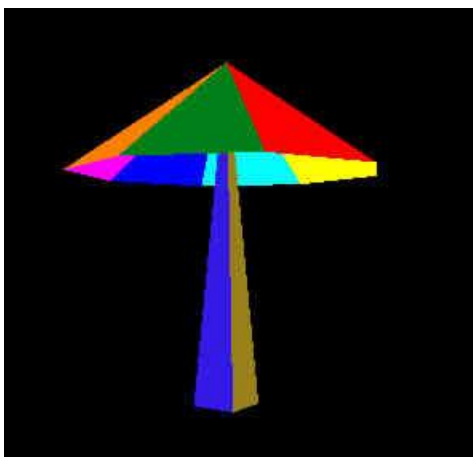
Extra Activity 1



1. Refer the code in Chapter 2, modify the square so that it displays a hexagon (see the above figure).
2. Try to change the sequence of your vertex and observe the change in shape.

Extra Activity 2: Umbrella – Creating objects based on triangles

With what you have learned so far, you are to modify the `MyTriangles::draw()` function to produce the following object shown in two different views (you can set by using your favorites colors)



* END OF LAB *